

# 1. PORTADA

---

**Asignatura:** Programacion IV

**Estudiante:** Juan Andres Revollo Gutierrez

**Docente:** Ing. Jared Lopez Leano

**Fecha:** 2026-09-28

## 2. PUNTO 1: DISEÑO E IMPLEMENTACIÓN DEL CRUD

---

La entidad Producto representa los artículos gestionados en el sistema de inventario (Pingux POS). Cada producto cuenta con los campos obligatorios establecidos en el modelo:

- código: CharField único (max. 50). Identificador interno del producto.
- nombre: CharField (max. 150). Nombre comercial o descriptivo.
- descripción: TextField (opcional). Detalles adicionales del producto.
- categoría: CharField (max. 100). Clasificación del producto para reportes y filtros.
- precio: DecimalField (max\_digits=10, decimal\_places=2). Precio unitario en moneda local (Bs.). Se valida que no sea negativo.
- cantidad\_existente: IntegerField (default=0). Stock disponible en el almacén. Se valida que no sea negativo.
- stock\_minimo: IntegerField (default=5). Umbral para considerar stock bajo. Se valida que no sea negativo.
- estado: BooleanField (default=True). Indica si el producto está activo (visible en operaciones y reportes).
- fecha\_registro: DateTimeField (auto\_now\_add=True). Fecha y hora de alta del producto.

Fragmento clave - inventario/models.py (validaciones en modelo):

```
from django.core.validators import MinValueValidator
from django.db import models

class Producto(models.Model):
    codigo = models.CharField(max_length=50, unique=True, verbose_name="Codigo unico")
    nombre = models.CharField(max_length=150, verbose_name="Nombre")
    descripcion = models.TextField(blank=True, default="", verbose_name="Descripcion")
    categoria = models.CharField(max_length=100, verbose_name="Categoria")
    precio = models.DecimalField(
        max_digits=10, decimal_places=2, verbose_name="Precio",
        validators=[MinValueValidator(0, "El precio no puede ser negativo.")]
    )
    cantidad_existente = models.IntegerField(
        default=0, verbose_name="Cantidad disponible",
        validators=[MinValueValidator(0, "La cantidad no puede ser negativa.")]
    )
    stock_minimo = models.IntegerField(
        default=5, verbose_name="Stock minimo",
        validators=[MinValueValidator(0, "El stock minimo no puede ser negativo.")]
    )
```

```
)
estado = models.BooleanField(default=True, verbose_name="Activo")
fecha_registro = models.DateTimeField(auto_now_add=True, verbose_name="Fecha de registro")
```

Para garantizar la integridad de datos en el servidor, se implemento un formulario centralizado con validaciones explicitas:

Fragmento clave - inventario/forms.py (ProductoForm):

```
def clean_codigo(self):
    codigo = self.cleaned_data.get("codigo", "").strip()
    if not codigo:
        raise forms.ValidationError("El codigo unico es obligatorio.")
    conflictos = Producto.objects.filter(codigo__iexact=codigo)
    if self.instance.pk:
        conflictos = conflictos.exclude(pk=self.instance.pk)
    if conflictos.exists():
        raise forms.ValidationError("Ya existe un producto con ese codigo unico.")
    return codigo

def clean_precio(self):
    precio = self.cleaned_data.get("precio")
    if precio is not None and precio < 0:
        raise forms.ValidationError("El precio no puede ser negativo.")
    return precio

def clean_cantidad_existente(self):
    cantidad = self.cleaned_data.get("cantidad_existente")
    if cantidad is not None and cantidad < 0:
        raise forms.ValidationError("La cantidad no puede ser negativa.")
    return cantidad

def clean_stock_minimo(self):
    stock_minimo = self.cleaned_data.get("stock_minimo")
    if stock_minimo is not None and stock_minimo < 0:
        raise forms.ValidationError("El stock minimo no puede ser negativo.")
    return stock_minimo
```

Las operaciones CRUD se ejecutan via AJAX en el panel. La vista valida con ProductoForm y responde en JSON:

Fragmento clave - inventario/views.py (guardar\_producto):

```
@csrf_exempt
def guardar_producto(request):
    if request.method != "POST":
        return JsonResponse({"error": "Metodo no permitido"}, status=405)

    prod_id = request.POST.get("id")
    producto = get_object_or_404(Producto, id=prod_id) if prod_id else None
    form = ProductoForm(request.POST, instance=producto)
    if not form.is_valid():
        return JsonResponse({"error": "; ".join(
            f"{campo}: {mensajes[0]}" for campo, mensajes in form.errors.items()
        )}, status=400)

    form.save()
    Categoria.objects.get_or_create(nombre=form.cleaned_data["categoria"].strip())
    return JsonResponse({"ok": True, "status": "ok"})
```

La ultima linea sincroniza el catalogo: si la categoria escrita en el formulario es nueva, queda registrada y pasa a estar disponible en el combo de los siguientes productos.

Gestion de categorias (amplia el RF-01): el catalogo de categorias es una entidad propia (modelo Categoria, tabla categorias) que se administra desde el panel del sistema, no desde el chatbot. La migracion 0004 creo la tabla y la poble con las categorias que ya usaban los productos, de modo que no se perdiera ninguna al actualizar el sistema.

Borrado logico y fisico (RF-04): la vista eliminar\_producto soporta los dos modos. Por defecto el borrado es logico, es decir desactiva el producto con estado=False sin perder la fila, que es lo seguro para un POS porque conserva el historial de ventas; el modo fisico elimina definitivamente el registro. El boton de eliminar pide confirmar el modo de forma explicita, y los productos desactivados se pueden revisar con el boton "Ver inactivos" y reactivar desde el formulario.

Fragmento clave - inventario/views.py (eliminar\_producto):

```
@csrf_exempt
def eliminar_producto(request, pk):
    if request.method != "POST":
        return JsonResponse({"error": "Metodo no permitido"}, status=405)
    producto = get_object_or_404(Producto, pk=pk)
    modo = (request.POST.get("modo") or "logico").strip().lower()

    if modo == "fisico":
        producto.delete()
        return JsonResponse({"ok": True, "modo": "fisico", "status": "ok"})
    if modo != "logico":
        return JsonResponse({"error": "Modo invalido. Usa 'logico' o 'fisico'."}, status=400)

    producto.estado = False
    producto.save(update_fields=["estado"])
    return JsonResponse({"ok": True, "modo": "logico", "status": "ok"})
```

Los cuatro botones de consulta rapida exigidos por la rubrica estan en el panel y devuelven los datos exactos, siempre con la tabla completa y sin recortes: "Stock critico" y "Producto mas caro" van por POST a /api/chat/; "Valor total" responde la cifra y ademas lista el detalle por producto; y "Reporte: agotados" va por GET a /api/reporte/?tipo=agotados. Los botones que dependen de la IA muestran un estado "Generando..." inmediato, porque el analisis del modelo puede tardar alrededor de 20 segundos.

Fragmento clave - inventario/views.py (eliminar\_categoria):

```
@csrf_exempt
def eliminar_categoria(request, pk):
    categoria = get_object_or_404(Categoria, pk=pk)
    en_uso = Producto.objects.filter(categoria__iexact=categoria.nombre).count()
    if en_uso:
        return JsonResponse({
            "error": f"No se puede eliminar '{categoria.nombre}': {en_uso} producto(s) la estan usando."
        }, status=400)
    categoria.delete()
    return JsonResponse({"ok": True})
```

Decision de diseno: Producto.categoria se mantiene como texto y Categoria funciona como catalogo de apoyo. Asi los filtros y los 8 reportes ya validados no cambian, y a la vez se puede elegir de una lista, agregar y borrar categorias. Renombrar una categoria actualiza en cascada los productos que la usaban.

Diseno de la pantalla: el requerimiento pide una pantalla del chatbot y otra del sistema, por lo que el panel se organizo en dos columnas. A la izquierda queda el sistema (metricas y catalogo de productos) y a la derecha queda unicamente el chatbot (accesos rapidos, conversacion y entrada de texto). Lo que no es consulta del dia a dia se abrio en modales accesibles desde la barra superior: Reportes (modal ancho

de 1040px para que las tablas se lean completas), Categorías (alta y baja) e Historial. Así el usuario trabaja sobre modales grandes en vez de alargar la página.

Además, la pantalla principal se dimensiona para no obligar a hacer scroll: el main es un contenedor flex de columna, el grid del workspace usa flex: 1 con min-height: 0, y tanto la tabla de productos como la conversación tienen su propio scroll interno. Así, con cualquier alto de ventana, métricas, tabla y chatbot entran en una sola pantalla y el scroll queda solo dentro de cada área. En móvil (hasta 1024px) las columnas se apilan y vuelve el scroll normal de la página.

El panel de reportes muestra el estado "Generando reporte..." de inmediato, porque el análisis de Ollama puede tardar alrededor de 20 segundos; antes el usuario no recibía ninguna señal visual y creía que el botón no funcionaba.

Reporte predefinidos implementados (RF-06): la aplicación cuenta con 8 reportes, registrados mediante un diccionario tipo Strategy en reportes.py. Cada reporte genera una tabla Markdown, solicita una breve explicación a Ollama (con fallback) y guarda el resultado en ConsultalA.

Reportes disponibles (tipo): - todos: Listado completo de productos activos. - mas\_caro: Top 5 productos con mayor precio. - mas\_barato: Top 5 productos con menor precio. - pocas\_existencias: Productos activos con cantidad > 0 y <= stock\_minimo. - agotados: Productos activos con cantidad\_existente == 0. - por\_categoria: Inventario filtrado por categoría (opcional parámetro categoría). - valor\_total: Resumen de productos activos, unidades totales y valor económico estimado (Bs.). - mayor\_cantidad: Top 5 productos con mayor cantidad en stock.

### 3. PUNTO 2: INTEGRACION CON IA LOCAL MEDIANTE OLLAMA

---

La integración se realiza contra el servidor local de Ollama, sin depender de servicios en la nube. La configuración se centraliza en variables de entorno o archivo .env (cargado con python-dotenv), con valores por defecto seguros:

Fragmento - config/settings.py:

```
OLLAMA_API_URL = os.environ.get("OLLAMA_API_URL", "http://localhost:11434/api/generate")
OLLAMA_MODELO = os.environ.get("OLLAMA_MODELO", "pos-inventario-bot")
OLLAMA_TIMEOUT = float(os.environ.get("OLLAMA_TIMEOUT", "120"))
```

El cliente HTTP usa la librería httpx con un timeout amplio (120 s) debido a que el modelo se ejecuta en CPU. El endpoint /api/generate recibe un prompt que combina el contexto JSON y la pregunta del usuario. El SYSTEM estricto (solo asistente de inventario, sin inventar datos) se define en Modelfile.txt y se registra con:

```
ollama create pos-inventario-bot -f Modelfile.txt
```

Servicio que arma el contexto JSON con los productos activos (inventario/ollama\_client.py):

```
def build_context(limite=10):
    activos = Producto.objects.filter(estado=True)
    productos = [
        {
            "codigo": p.codigo,
            "nombre": p.nombre,
            "categoria": p.categoria,
            "precio_bs": float(p.precio),
```

```

        "stock": p.cantidad_existente,
        "stock_minimo": p.stock_minimo,
        "estado": _estado_stock(p),
    }
    for p in activos.order_by("categoria", "nombre")[:limite]
]
return {
    "total_productos_activos": activos.count(),
    "unidades_en_stock": sum(p.cantidad_existente for p in activos),
    "valor_inventario_bs": round(float(sum(p.cantidad_existente * p.precio for p in activos)), 2),
    "productos_con_stock_critico": activos.filter(
        cantidad_existente__lte=F('stock_minimo'), cantidad_existente__gt=0
    ).count(),
    "productos_agotados": activos.filter(cantidad_existente=0).count(),
    "muestra_productos": productos,
}

```

Ejemplos de preguntas del chat y respuestas reales obtenidas:

Pregunta 1: "Cual es el producto mas caro?" Respuesta (deterministica, instantanea, sin llamar al modelo):

El producto **mas caro** es **Vino tinto** (PRO-056) de la categoria *General*, con un precio de **Bs. 185.00**.

Ranking de precios:

| Producto                         | Stock | Precio     | Estado  |
|----------------------------------|-------|------------|---------|
| Vino tinto                       | 5     | Bs. 185.00 | Bajo    |
| Queso San Javier 1kg             | 0     | Bs. 38.00  | Agotado |
| Cafe Nescafe 200g                | 0     | Bs. 35.00  | Agotado |
| Huevos Granja Maple 30 un        | 0     | Bs. 28.00  | Agotado |
| Desodorante Rexona Aerosol 150ml | 0     | Bs. 22.00  | Agotado |

Pregunta 2: "Cuantos productos hay en el almacen?" Respuesta (deterministica, instantanea):

Hay **51** productos activos en el almacen (41 agotados).

Pregunta 3: "Resume en pocas lineas el estado general del inventario" Respuesta real generada por pos-inventario-bot (via Ollama, ~32 s en CPU):

El inventario general del sistema de Pingux POS esta bien controlado. Todos los productos estan en stock optimo, con excepcion de un producto que esta agotado: el azucar Guabira 1kg.

Pregunta 4: "Lista 3 productos que esten agotados" Cuando Ollama agota el tiempo de espera (timeout 120 s), el sistema aplica el fallback local y responde con datos de la base de datos, sin depender del modelo:

Hay **5** producto(s) con stock bajo o agotado:

| Producto                     | Stock | Precio   | Estado  |
|------------------------------|-------|----------|---------|
| Azucar Guabira 1kg           | 0     | Bs. 6.00 | Agotado |
| Fideos Famosa Macarron 400g  | 0     | Bs. 4.50 | Agotado |
| Fideos Famosa Spaguetti 400g | 0     | Bs. 4.50 | Agotado |
| Harina de Trigo 1kg          | 0     | Bs. 6.50 | Agotado |
| Lentejas Grano de Oro 500g   | 0     | Bs. 8.00 | Agotado |

Manejo de excepciones si Ollama esta apagado (inventario/ollama\_client.py):

```
class ModeloNoRegistrado(Exception):
    """Se lanza cuando Ollama responde 404: el modelo aun no fue creado."""
    pass

def consultar_ollama(pregunta, contexto_json=None, options=None):
    ...
    try:
        with httpx.Client(timeout=TIMEOUT_SEGUNDOS) as client:
            respuesta = client.post(OLLAMA_API_URL, json=payload)
            respuesta.raise_for_status()
            return respuesta.json().get("response", "").strip()
    except httpx.HTTPStatusError as e:
        if e.response.status_code == 404:
            raise ModeloNoRegistrado(MODELO) from e
        return None
    except Exception as e:
        logger.warning("No se pudo consultar a Ollama (%s): %s", MODELO, e)
        return None
```

En la capa de chat (inventario/ia.py) se Manejan los tres casos:

- Modelo no registrado (404): se captura ModeloNoRegistrado y se responde con un aviso que indica el comando ollama create, mas los datos locales.
- Servidor apagado o error de red: se captura Exception y se genera una respuesta de respaldo con los productos del contexto.
- Timeout: tras 120 s la peticion falla, se registra el warning y se responde con el fallback local.

En todos los casos la consulta se registra en ConsultasIA, por lo que la UI muestra un historial completo.

## 4. PUNTO 3: CALIDAD, PATRONES Y DOCUMENTACION

Patrones aplicados en el codigo:

- Strategy (inventario/reportes.py): cada tipo de reporte es un algoritmo intercambiable. El diccionario REPORTES y la funcion generar\_reporte(tipo) seleccionan la estrategia en tiempo de ejecucion, sin

condicionales encadenados. Esto permite agregar nuevos reportes sin modificar la logica existente.

```
REPORTES = {
    "todos": reporte_todos,
    "mas_caro": reporte_mas_caro,
    "mas_barato": reporte_mas_barato,
    "pocas_existencias": reporte_pocas_existencias,
    "agotados": reporte_agotados,
    "por_categoria": reporte_por_categoria,
    "valor_total": reporte_valor_total,
    "mayor_cantidad": reporte_mayor_cantidad,
}

def generar_reporte(tipo, categoria=None):
    """Patron Strategy: selecciona en tiempo de ejecucion el algoritmo de
    reporte correspondiente al tipo solicitado."""
    if tipo not in REPORTES:
        raise ValueError(f"Reporte desconocido: {tipo}")
    if tipo == "por_categoria":
        return reporte_por_categoria(categoria)
    return REPORTES[tipo]()
```

- Facade (inventario/ollama\_client.py): consultar\_ollama() encapsula todo el detalle de la API REST de Ollama (payload, timeout, serializacion, manejo de errores HTTP y fallback silencioso). El resto del sistema solo ve una funcion simple.

Resultado de las pruebas unitarias: se implemento una suite de 53 pruebas automatizadas en inventario/tests.py que cubren modelos, formularios, endpoints CRUD, gestion de categorias (alta, duplicados ignorando mayusculas, renombrado en cascada y borrado bloqueado si esta en uso), los cuatro botones de consulta rapida, los ocho reportes con el criterio unico de productos activos, cliente Ollama (incluyendo 404 y error de conexion) y chat (incluye el descarte de respuestas truncadas del modelo). La suite corre en menos de un segundo porque el cliente de Ollama va simulado con mock, de modo que las pruebas no dependen de que el servidor de IA este encendido.

Comando de ejecucion:

```
python manage.py test
```

Resultado obtenido:

```
Found 29 test(s).
System check identified no issues (0 silenced).
Ran 53 tests in 0.172s

OK
```

Referencia al requirements.txt: el archivo fija las dependencias del proyecto para garantizar reproducibilidad.

```
Django==5.2.17
ollama==0.6.2
python-dotenv==1.2.3
markdown==3.11
xhtml2pdf==0.2.21
```

Instalacion segun README.md: el README documenta el procedimiento completo. En resumen:

```
python -m venv venv
source venv/bin/activate
cp .env.example .env
pip install -r requirements.txt
python manage.py makemigrations inventario
python manage.py migrate
python manage.py runserver 0.0.0.0:8000
```

Ademas, el archivo `.env.example` permite ajustar la configuracion de Ollama (`OLLAMA_API_URL`, `OLLAMA_MODELO`, `OLLAMA_TIMEOUT`) ni la de Django (`SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS`) sin tocar el codigo fuente. Los tres archivos de documentacion del entregable son: `README.md` (guia rapida), `DOCUMENTACION.md` (manual tecnico completo con endpoints, validaciones, pruebas y solucion de problemas) y `OPENCODE.md` (convenciones y prohibiciones para asistentes de IA que trabajen en el repositorio).

Correccion aplicada durante la auditoria final: el reporte "valor total" sumaba las unidades de todos los productos, incluidos los desactivados, mientras que el valor economico y el conteo de productos ya filtraban por `estado=True`. Esa incoherencia hacia que las cifras no cuadraran con la tabla. Se unifico el criterio en un unico `queryset` activo y se agrego una prueba de regresion que lo verifica para los ocho reportes.

## 5. REFLEXION TECNICA FINAL

---

Las principales dificultades durante la sincronizacion con la API local de Ollama fueron:

- Latencia del modelo en CPU: la primera consulta en frio tardo alrededor de 70 s, y las siguientes entre 13 y 32 s. Se resolvió con un `timeout` de 120 s y reduciendo el tamaño del contexto enviado al modelo (muestra acotada de productos). Para las preguntas frecuentes (conteo, precios, valor total, ranking) se implementó una capa de intents deterministas que responde en menos de 1 segundo sin invocar al modelo.
- Errores de red y apagado del servidor: cuando Ollama no está disponible, la petición falla con `ConnectError` o `timeout`. Se solucionó con un `try/except` que captura cualquier excepción y devuelve un fallback local, de modo que el usuario siempre recibe una respuesta con datos reales de la base de datos.
- Modelo no registrado (HTTP 404): al iniciar en un entorno limpio, Ollama responde 404 porque `pos-inventario-bot` aun no fue creado. Se detectó este caso particular y se creó la excepción `ModeloNoRegistrado`, que permite diferenciar "modelo no creado" de "modelo no disponible" y mostrar un mensaje con la instrucción exacta `ollama create pos-inventario-bot -f Modelfile.txt`.
- Respuestas largas o truncadas: al pedir listas de productos, el modelo a veces devolvía tablas Markdown incompletas o símbolos de moneda inconsistentes. Se mejoró el `SYSTEM` en `Modelfile.txt` (temperatura 0.1, `num_predict` limitado, respuesta estructurada) y se priorizó el fallback determinista para preguntas de listados.

El uso de variables de entorno y `.env` permitio desacoplar la configuracion de Ollama del codigo, facilitando el despliegue en diferentes equipos. La combinacion de validaciones en modelo y formulario garantiza la integridad de los datos, y las pruebas unitarias verifican que el sistema se mantiene estable ante cambios futuros.

## USO DE OPENCODE COMO ASISTENTE DE DESARROLLO

Durante el desarrollo de Pingux POS, se utilizo OpenCode directamente desde la terminal MiniOS (Debian 12) como asistente de codificacion.

### Sesiones y Prompts Clave:

1. **Generacion de validaciones en forms.py:** - *Prompt:* "Ayudame a estructurar los metodos `clean_codigo` y `clean_precio` en `ProductoForm` para asegurar que no existan codigos duplicados (case-insensitive) ni valores negativos." - *Impacto:* OpenCode genero de forma rapida la logica de exclusion de la instancia actual en caso de edicion, optimizando el tiempo de desarrollo.
2. **Implementacion del patron Strategy para reportes:** - *Prompt:* "Disena un diccionario de estrategias en Python para gestionar 8 tipos de reportes predefinidos de inventario sin usar multiples condicionales if-else." - *Impacto:*Codigo limpio, mantenible y escalable.
3. **Manejo de excepciones HTTP y Fallback para Ollama:** - *Prompt:* "Escribe un bloque try-except usando `httpx` que capture errores 404 (modelo no registrado) y timeouts, devolviendo un fallback con datos de SQLite." - *Impacto:* Garantizo que la aplicacion nunca se caiga si el servidor local de Ollama esta apagado o saturado.